

Program Optimization for Verifiable Problems

Allen Liu
Winson Wang
Jonathan Xu

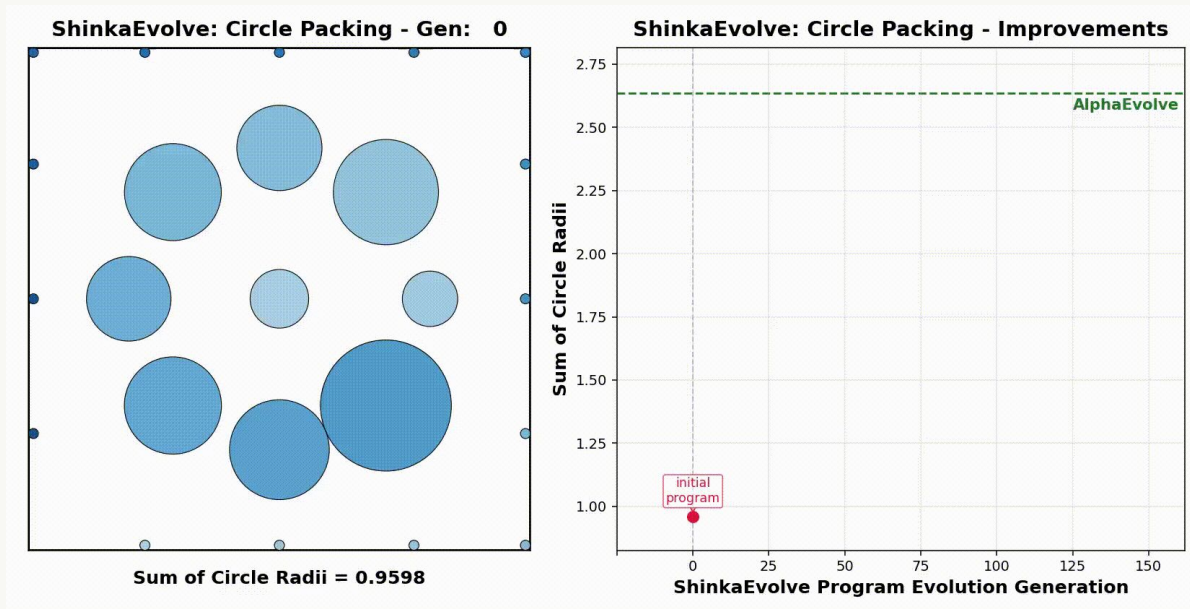
Motivation

Motivation

- LLMs are good at **generating** solutions, but bad at **verifying** them
- One solution: evaluate LLMs against a **strictly verifiable** problem

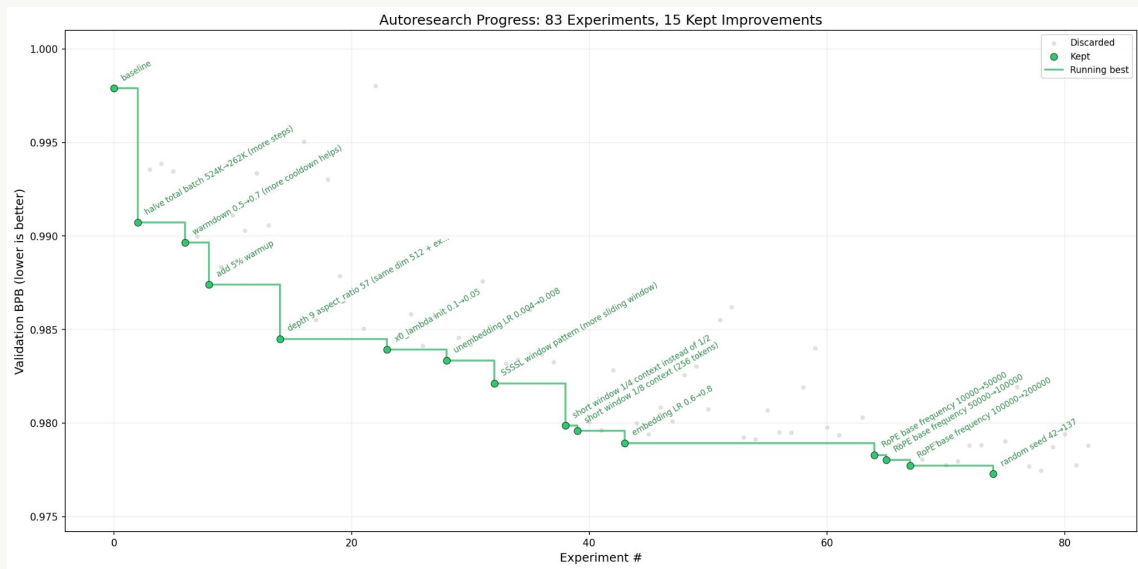
Many such examples:

- Kernel speed
- Circle packing
- Coding benchmark



AlphaEvolve → Autoresearch

- Put LLMs in a loop with a deterministic evaluator, have them iteratively to beat the “best record” until SOTA achieved



Part I: Gradient optimization

Circle packing (CP26)

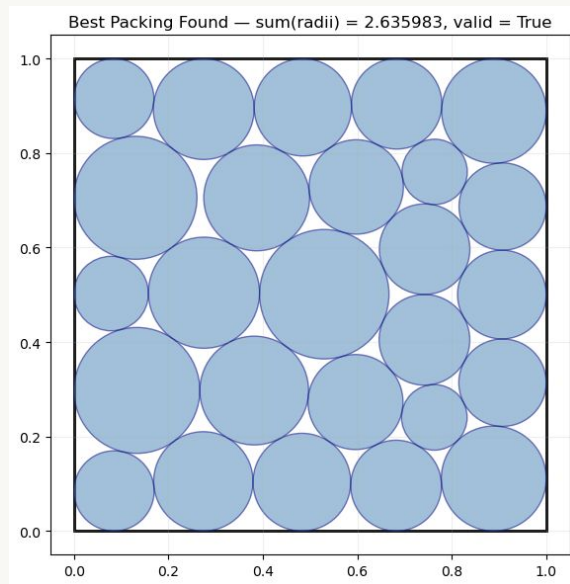
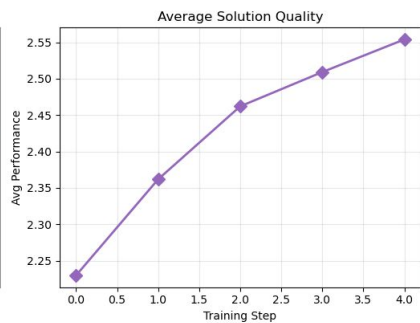
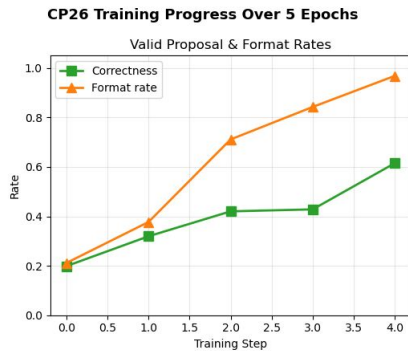
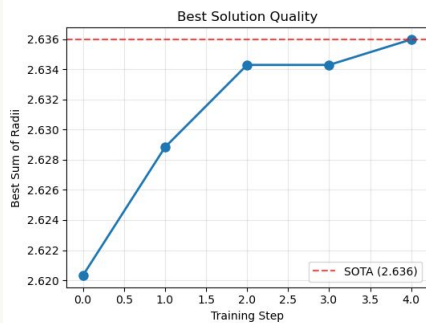
- Place 26 circles in a unit square
- Maximize total sum of radii

SOTA solution: TTT-Discover

Method	Model	$n = 26$ ($\uparrow$)	$n = 32$ ($\uparrow$)
AlphaEvolve [50]	Gemini-2.0 Pro + Flash	2.635862	2.937944
AlphaEvolve V2 [14]	Gemini-2.0 Pro + Flash	2.635983	2.939572
ShinkaEvolve [37]	Ensemble (see caption)	2.635982	n/a
ThetaEvolve [78]	R1-Qwen3-8B	2.635983	n/a
TTT-Discover	Qwen3-8B	2.635983	2.939572

Implementation

- Train open-source model (**gpt-oss-120b**) with LoRA
- Generate 64 rollouts per batch, then fine-tuned to upweight best solutions
- Save a bank of good solutions, re-use on every next batch



Key insights

- Upweight the best solutions exponentially - only care about best performance
- A small base model won't get good results with RL
- Context might matter more than weights

Problems:

- Very compute expensive (cost ~\$30 for 5 batches)
- RL can be very unstable/inconsistent

Part II: Prompt optimization

Implementation

GEPA `optimize_anything`

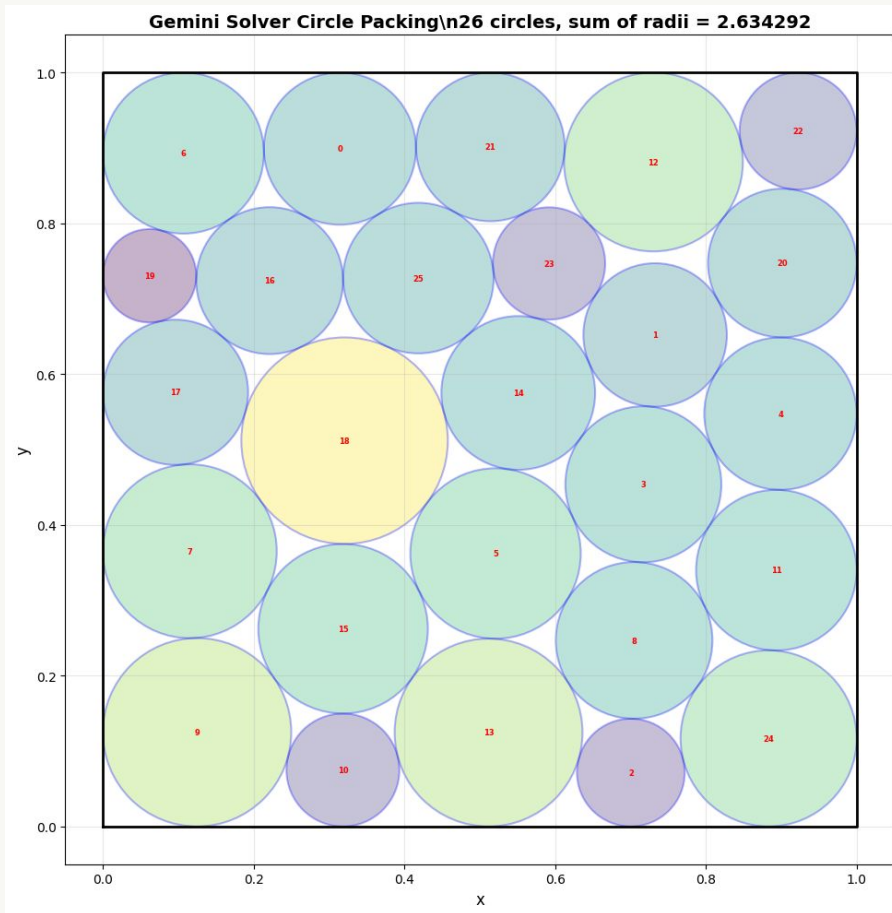
- A reflection LLM proposes edits to the solver program
- Solver program is executed locally and calls LLM function
- Evaluator runs the candidate program and verifies constraints

Why GEPA?

- Avoids training weights and instead performs structured search over programs
- Improves compute efficiency of finding best solution (can use very small LLMs)
 - Gemini 2.5 Flash Lite, o3-mini, etc.

Results

w/ Gemini 2.5 Flash

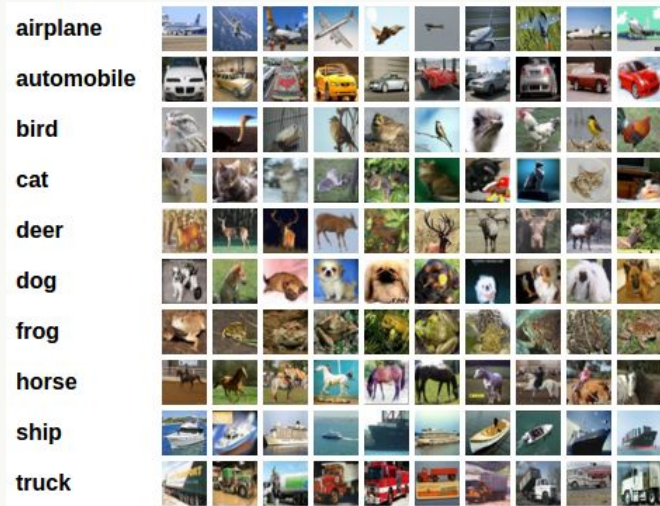


Part III: AirBench

AirBench

- Train a neural network to 94% accuracy on CIFAR-10 as fast as possible

Script	Mean accuracy	Time	PFLOPs
airbench94_muon.py	94.01%	2.59s	0.29



Implementation

- Autoresearch loop
- Add success/failure data to **memory.md** so agent can learn from context
- Use **gemini-3.1-flash-lite-preview** + A100-40GB GPU on Modal
- Start from the best incumbent script in AirBench repo

We also tried a GEPA Loop:

- Multi-agent approach - have 1 coordinator control several worker agents
- History-aware prompting - inject recent eval history, best rows, and lessons into prompts so the proposer could see what had already failed

Results Autoresearch



Results GEPA

12 runs, 33 GEPA proposals generated across ~6 weeks (March 6–22)

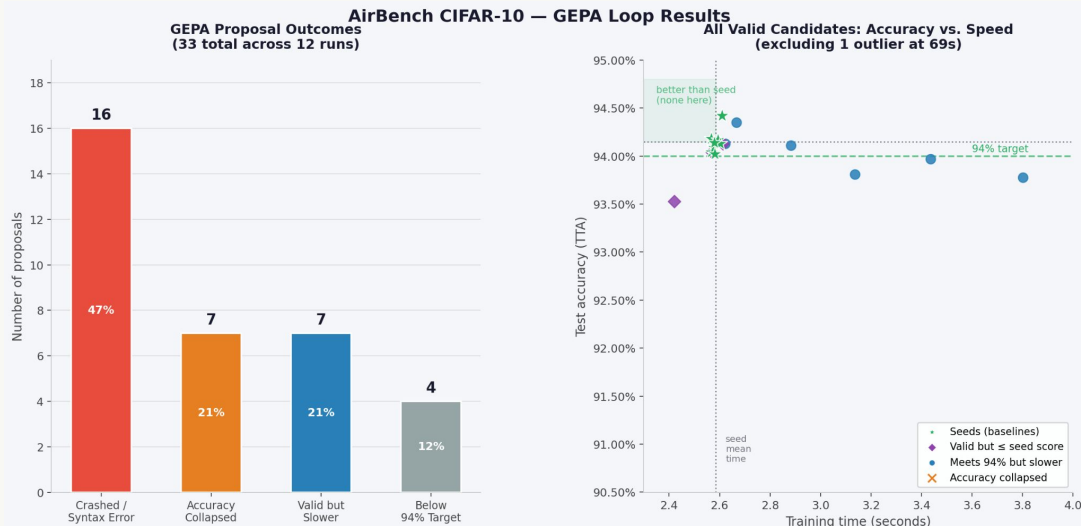
0 proposals ever beat the seed — best score in every run came from the starting code

Nearly half (47%) crashed outright with runtime or syntax errors

21% met the 94% accuracy target but ran slower than the baseline

21% produced valid-looking code that broke accuracy below 94%

The green "better than seed" region on the scatter plot remained completely empty



Part IV: Parameter Golf

“Make the best mini model possible”

The Competition

OpenAI's Model Craft Challenge: train the **best language model that fits in 16 MB** and trains in under 10 minutes on 8×H100 GPUs.

Constraint: Artifact size $\leq 16,000,000$ bytes (code + compressed weights)

Evaluation: Performance on FineWeb validation set

16 MB

Max Artifact Size

10 min

Training Budget (8×H100)

BPB

Bits Per Byte — our metric

What is BPB?

"How often does the model get surprised?"

The more predictable the text, the less surprised the model is — and the lower the BPB.

No surprise	Some surprise	Total surprise	How it's calculated						
"The cat sat on the ____" → <i>mat</i> Model is confident. Very few bits needed. Low BPB	"She opened the door and saw a ____" → <i>stranger / dog / light ...</i> Multiple plausible answers. Medium BPB	"The stock price moved ____" → <i>could be anything</i> Highly unpredictable. Many bits needed. High BPB	BPB = $-\log_2(p)$ p = probability the model assigned to the correct next word <table><tr><td>p = 0.9 (Very confident)</td><td>→ 0.15</td></tr><tr><td>p = 0.5 (Uncertain)</td><td>→ 1.0</td></tr><tr><td>p = 0.1 (Surprised)</td><td>→ 3.3</td></tr></table>	p = 0.9 (Very confident)	→ 0.15	p = 0.5 (Uncertain)	→ 1.0	p = 0.1 (Surprised)	→ 3.3
p = 0.9 (Very confident)	→ 0.15								
p = 0.5 (Uncertain)	→ 1.0								
p = 0.1 (Surprised)	→ 3.3								

Our goal: train a model that predicts text so well it consistently gets a low BPB

Approach 1: Shallow Recurrence

What is Shallow Recurrence?

Instead of a standard transformer that processes each token once per layer, shallow recurrence runs multiple passes over the same layers — reusing parameters to get more "compute" out of fewer weights.

This is especially valuable in the parameter-constrained setting: we get depth without paying the parameter cost of extra layers.

Why it helps here

- Fewer parameters, same effective depth
- Fits comfortably within 16 MB
- Improved BPB from 1.601 → 1.240 over baseline

What We Did

1

Start with baseline architecture

9-layer GPT transformer as the foundation, 1024-token vocabulary.

2

Add recurrent passes

Applied `pr1493_shallow_recurrence_pass_scaled` — same weights, multiple forward passes per token.

3

Achieved best result

Run 8 produced BPB 1.240 — a 23% improvement over the baseline.

Key insight: Recurrence lets you extract more intelligence from fewer parameters — perfect for a 16 MB constraint.

Approach 2: Pushing Further

Runs 9–10

Scaled-Up Recurrence · Run 9

Best BPB: 1.325

The idea

If shallow recurrence works, what if we scale the model up to ~25 MB and rely on zlib compression to bring it back under 16 MB?

What happened

- Best observed BPB was 1.325 mid-training
- Final logged BPB regressed to 1.501 — model too large to converge in 10 min
- Artifact produced at 23.2 MB — exceeded size limit

GPTQ 15 MB · Run 10

Best BPB: 1.300

The idea

Use GPTQ — a post-training quantization method — to aggressively compress a well-trained model down to 15 MB while preserving as much quality as possible.

What happened

- Best observed BPB was 1.300 mid-training
- Final logged BPB was 1.348 — gap suggests run needed more time to converge
- Most promising avenue if given more training budget

Key lesson: Bigger is not always better within a fixed time budget — convergence speed matters as much as model capacity.

Our Attempts

Baseline

Run 2

Best BPB

1.601

Standard GPT-style transformer on 8×H100. End-to-end pipeline confirmed working. Sets our starting point.

Shallow Recurrence

Runs 6–8

Best BPB

1.240

New architecture with recurrent passes. After fixing tokenizer and FA3 import issues, achieved our best result so far.

Scaled-Up Recurrence

Run 9

Best BPB

1.325

Pushed the same architecture to a larger ~25 MB model class. Regressed at final logging — too large to train well in the time budget.

GPTQ 15 MB

Run 10

Best BPB

1.300

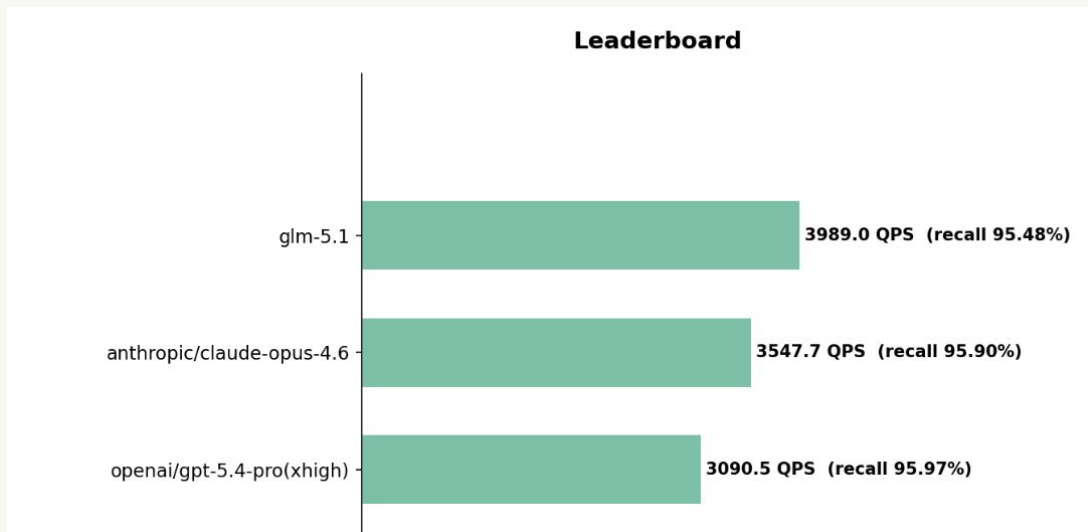
Aggressive quantization targeting a 15 MB artifact. Promising intermediate BPB but final logged BPB was 1.348 — gap suggests more training time needed.

Best valid result: Shallow Recurrence (Run 8) — BPB 1.240, artifact within 16 MB

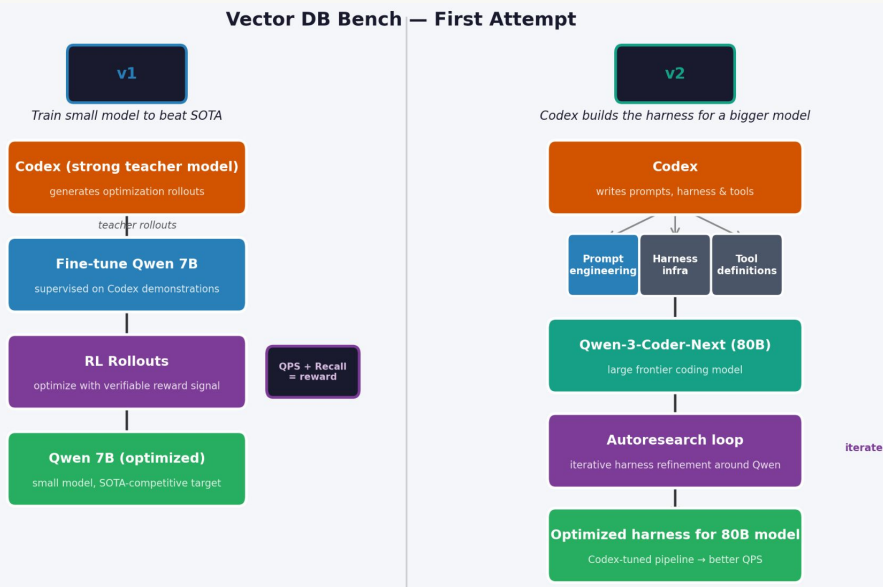
Part V: VectorDBBench

Vector DB Bench

- Benchmarking tool for evaluating vector databases
- Tests performance on high-dimensional embedding search
- Measures key metrics: latency, throughput, and recall
- QPS (Queries Per Second): number of search requests the system can handle each second



Implementation (1st try)



v1:

- Train a small Qwen 7B model to beat SOTA performance
- Fine-tune Qwen on teacher rollouts from stronger model (Codex)
- Then perform RL rollouts

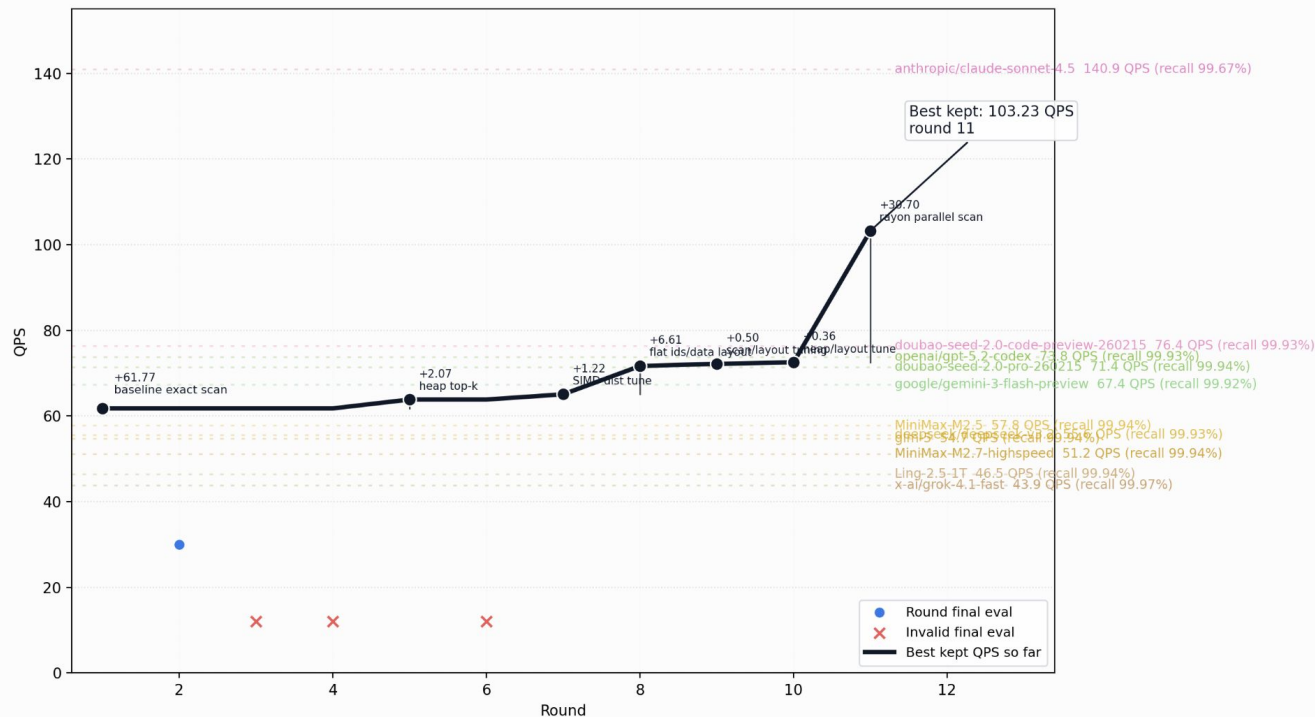
v2:

- Have Codex optimize the harness for an 80B model (Qwen-3-Coder-Next)
- Codex writes the prompting, harness, and tools for Qwen
- Autoresearch on the harness around Qwen

Results (1st try)

Dots: final evaluation per completed round. Black line: incumbent best-so-far. Red x markers indicate invalid final evaluations. Jump labels show kept deltas and inferred changes.

Continuous Qwen3-Coder-Next Run: Progress vs Comparable Reference Scores



Implementation (2nd try)

Codex “Superagent”(OpenAI's coding agent) is given a Rust HTTP server skeleton with stub vector DB functions

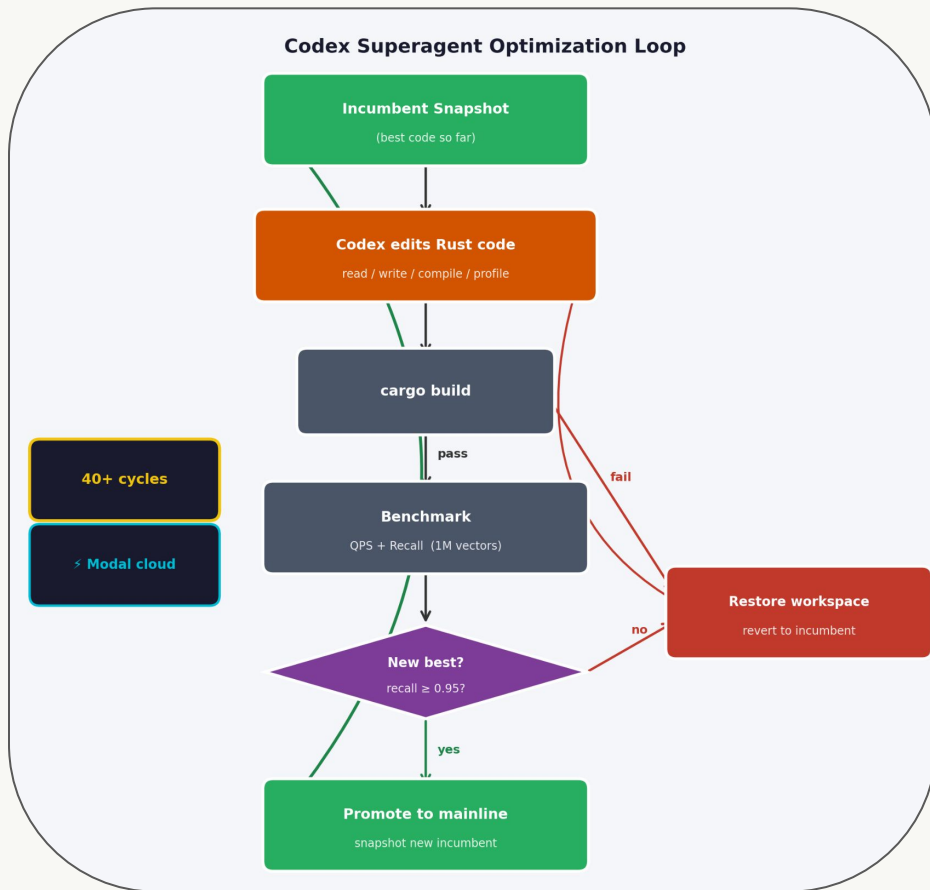
Runs inside a persistent optimization loop (the "Codex Superagent") on Modal cloud compute

Each cycle: Codex modifies code → cargo build → benchmark (QPS + recall) → compare vs. incumbent

If the new version beats the current best, it is promoted to the mainline snapshot; otherwise the workspace is restored

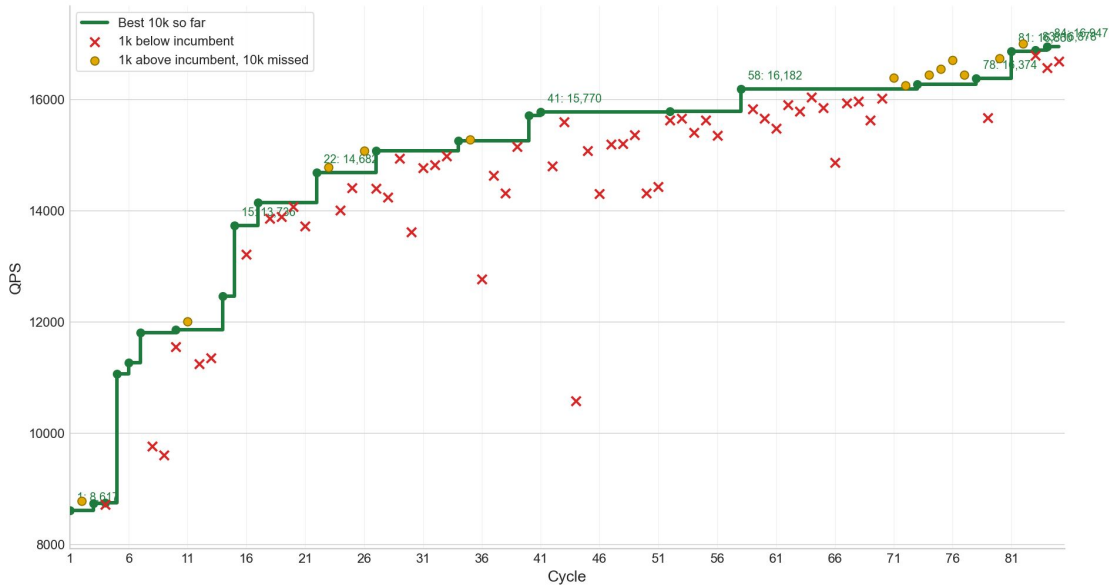
Loop runs for 40+ cycles; incumbent state persists across cycles so each attempt seeds from the best prior solution

Recall ≥ 0.95 is a hard constraint — invalid results are discarded regardless of QPS

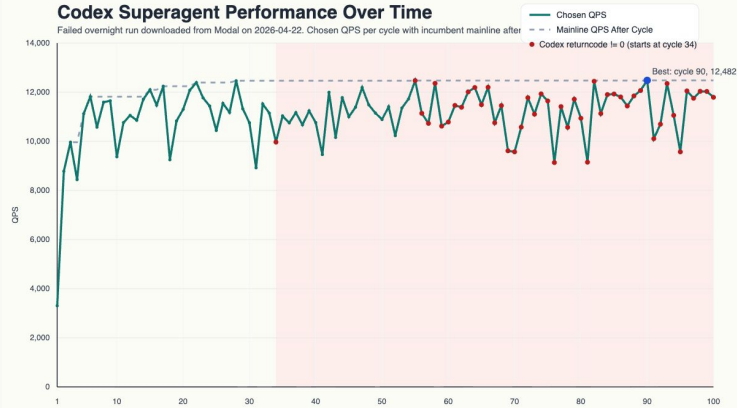


Results (2nd try)

Codex Superagent Progress Through Cycle 85



Codex Superagent Performance Over Time



Conclusions

Conclusions

- Verifier's law: Solving a task is proportional to how verifiable the task is
- Context + tooling often matters more than weight changes
- A bigger model can be exponentially better than smaller models
- “The models just want to learn” - Ilya Sutskever

“The models just want to optimize”

Thanking our “sponsors”



THINKING
MACHINES



Class Takeaways